DEPARTMENT OF
COMPUTER SCIENCE & ENGINEERING
Discover. Learn. Empower.

Experiment: 05

Student Name: Priyanshu Choudhary **UID: 23BCS12648**
Branch: BE CSE **Section/Group: KRG_3B**
Semester: 6th **Date of Performance: 27/02/26**
Subject Name: System Design **Subject Code: 23CSH-314**

1. Aim

To architect a highly scalable, real-time messaging application—comparable to platforms like WhatsApp or Facebook Messenger—capable of facilitating seamless one-to-one and group communication while ensuring minimal latency, robust availability, and dependable message transmission.

2. Objectives

- To comprehend the underlying architectural framework required for real-time messaging platforms.
- To determine essential functional prerequisites, including user authentication, interactive chat capabilities, and multimedia sharing.
- To establish critical non-functional parameters, primarily focusing on system availability, dependability, and strict latency limits.
- To critically evaluate the CAP theorem and its associated trade-offs within the context of messaging architectures.
- To construct robust RESTful and WebSocket API endpoints to power continuous chat communication.

3. Procedure

1. Conducted a comprehensive study of prominent real-world messaging infrastructures, such as WhatsApp and Facebook Messenger.
2. Defined the fundamental entities driving the system, specifically Users, Groups, and Messages.
3. Investigated the mechanics of persistent, real-time data exchange utilizing WebSocket protocols.

4. Gathered and categorized both the functional and non-functional system prerequisites.
5. Formulated the necessary API endpoints to handle secure user authentication and efficient message routing.
6. Assessed the immense storage capacities and scalability measures required to support massive, global user bases.
7. Balanced the requirement for high availability against the distributed data model of eventual consistency.

4. Functional Requirements

1. The platform must provide a secure portal for users to register and log in seamlessly.
2. The architecture must enable direct, one-to-one communication between individual users.
3. The system must support collaborative, multi-user group chat functionalities.
4. Users must possess the capability to transmit both standard text and rich media assets.
5. The database must persistently store and retrieve comprehensive chat histories for all interactions.
6. The user interface must provide real-time status updates through delivered and read receipts.
7. All message transmission and reception must occur instantaneously in real-time.

Core Entities of the System

- Users
- Groups
- Messages

API Design

1. User Registration API: POST /user/register
2. User Login API: POST /user/login

One-to-One Messaging A. Transmit Message (WebSocket): /messages/send B. Retrieve Active Chat List: GET /chat/{user_id} (Pagination Supported) C. Fetch Specific Chat History: GET /messages/{user_id}/{receiver_id}

Group Messaging A. Initialize Group: POST /groups/create B. Broadcast Group Message (WebSocket): /messages/send C. Load Group Chats: GET /groups/{group_id} (Pagination Supported) D. Integrate New Member: POST /groups/{group_id}/add E. Evict Member: DELETE /groups/{group_id}/remove

5. **Non-Functional Requirements**
    1. The infrastructure must be capable of processing extreme traffic volumes (ranging from millions to billions of daily messages), necessitating approximately 100 TB of storage capacity.
    2. High Availability must be prioritized in accordance with the CAP theorem's constraints.
    3. The system's data synchronization will adhere to an Eventual Consistency model.
    4. End-to-end message delivery latency should reliably fall within the 200-300 millisecond threshold.
    5. The network must be highly dependable, guaranteeing zero message loss and rigorously preventing packet drops.
    6. The entire foundation must be built upon a horizontally scalable, distributed architecture.
6. **High Level Design (HLD)**

    The overarching system architecture integrates several specialized components: Client Applications (covering both Mobile and Web interfaces), an API Gateway, an Authentication Service, dedicated Messaging Services functioning as WebSocket Servers, a Group Service, and a Media Service. Furthermore, the backend infrastructure relies on Message Queues, Distributed Databases, a high-speed Cache Layer (Redis), and Object Storage (BLOB) for managing media assets. Fundamentally, WebSockets ensure the immediate delivery of communications, while the distributed databases guarantee the persistent and safe storage of all messaging data.

7. **Learning Outcomes**
    - Successfully architected a highly scalable, real-time messaging infrastructure.
    - Accurately categorized the critical functional and non-functional system demands.
    - Constructed a cohesive and logical suite of RESTful and WebSocket API endpoints.
    - Gained a profound understanding of navigating the complex trade-offs between high availability and eventual consistency in distributed systems.